Listing 8.6 `langchain_load_splitting.py` (sections and output)

```
From langchain_community.document_loaders
                     ↪ import UnstructuredHTMLLoader    #1
from langchain.text_splitter import RecursiveCharacterTextSplitter
#previous code

loader = UnstructuredHTMLLoader(
                   "sample_documents/mother_goose.xhtml")   #2
data = loader.load    #3

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=25,     #4
    length_function=len,
    add_start_index=True,
)
documents = text_splitter.split_documents(data)

documents = [doc.page_content
               ↪ for doc in documents] [100:350]   #5

embeddings = [get_embedding(doc) for doc in documents]     #6
ids = [f"id{i}" for i in range(len(documents))]
###Output
Enter a search query (or 'exit' to stop): who kissed the girls and made
them cry?
How many top matches do you want to see? 3
Top Matched Documents:
ID:id233 TEXT: And chid her daughter,
        And kissed my sister instead of me. SCORE: 0.4…
```

#1 New LangChain imports
#2 Loads the document as HTML
#3 Loads the document
#4 Splits the document into blocks of text 100 characters long with a 25-character overlap
#5 Embeds only 250 chunks, which is cheaper and faster
#6 Returns the embedding for each document

Note in listing 8.6 that the HTML document gets split into 100-character chunks with a 25-character overlap. The overlap allows the document's parts not to cut off specific thoughts. We selected the splitter for this exercise because it was easy to use, set up, and understand.

Go ahead and run the `langchain_load_splitting.py` file in VS Code (F5). Enter a query, and see what results you get. The output in listing 8.6 shows good results given a specific example. Remember that we only embedded 250 document chunks to reduce costs and keep the exercise short. Of course, you can always try to embed the entire document or use a minor input document example.